

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x07 实验报告

更好的内存管理

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 帧分配器的内存回收

我们在 lab-4 的加分项中已经实现了帧分配器的内存回收，即用一个 `Vec<PhysFrame>` 作为回收的内存池，保存每次释放了的帧，然后每次 allocate 一个新的帧的时候先检查内存池种是否有可用的帧，如下

```
unsafe impl FrameAllocator<Size4KiB> for BootInfoFrameAllocator {
    fn allocate_frame(&mut self) -> Option<PhysFrame> {
        // FIXED: try pop from recycled frames
        self.used += 1;
        if let Some(reused_frame) = self.recycled.pop() {
            return Some(reused_frame);
        }
        // FIXED: if recycled is empty:
        //        try allocate from frames like before
        self.frames.next()
    }
}

impl FrameDeallocator<Size4KiB> for BootInfoFrameAllocator {
    unsafe fn deallocate_frame(&mut self, _frame: PhysFrame) {
        // FIXED: push frame to recycled
        self.recycled.push(_frame);
        self.used -= 1;
    }
}
```

2. 用户程序的内存统计

我们在本实验中只考虑堆内存，栈内存，以及 ELF 文件映射占用的内存，而不考虑页表等占用的内存。而前面两者我们已经实现了，因此我们这部分只需要考虑统计 ELF 文件映射占用的内存。

我们考虑在 `load_elf` 加载 ELF 文件的时候就进行记录，这里根据文档将 `load_elf` 修改为用一个 `Vec<PageRangeInclusive>` 记录所有加载的 ELF 文件映射的页表范围。

随后，我们在 `load_elf_code` 函数中，将返回的 `Vec<PageRangeInclusive>` 进行计算大小并求和，得到该部分占用的内存

```
fn load_elf_code(&mut self, elf: &ElfFile, mapper: MapperRef, alloc:
FrameAllocatorRef) {
    // FIXED: make the `load_elf` function return the code pages
    self.code =
        elf::load_elf(elf, *PHYSICAL_OFFSET.get().unwrap(), mapper, alloc,
true).unwrap();

    // FIXED: calculate code usage
    self.code_usage = self
        .code
        .iter()
        .map(|range| (range.count() as u64) * crate::memory::PAGE_SIZE)
        .sum();
}
```

随后, 我们修改 `print_process_list`, 加入对内存信息的统计, 由于我们实际使用的内存记录在 `used` 成员里面, 而且在每次 deallocate 的时候减去了, 因此我们这里计算只需要算 `used` 成员即可, 如下

```
// FIXED: print memory usage of kernel heap
let alloc = get_frame_alloc_for_sure();
let frames_used = alloc.frames_used();
let frames_total = alloc.frames_total();

let used = frames_used * PAGE_SIZE as usize;
let total = frames_total * PAGE_SIZE as usize;

output += &format_usage("Memory", used, total);
drop(alloc);
```

我们可以看到, 目前连续运行两次 factorial 的结果如下,

```
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |    4769 |  32.0 KiB | Running
 #  1 | #  0 | kernel       |    4827 |   2.0 MiB | Ready
Memory :   1.08 MiB /  44.50 MiB ( 2.43%)
Queue  : [1]
CPUs   : [0: 2]
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |  695366 |  32.0 KiB | Blocked
 #  3 | #  2 | /app/factor~1 |    7088 |  15.3 MiB | Running
 #  1 | #  0 | kernel       |  702511 |   2.0 MiB | Ready
Memory :  16.43 MiB /  44.50 MiB (36.93%)
Queue  : [1]
CPUs   : [0: 3]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [3] exited with code: 0
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |  730795 |  32.0 KiB | Blocked
 #  1 | #  0 | kernel       |  746605 |   2.0 MiB | Ready
 #  4 | #  2 | /app/factor~1 |    8658 |  15.3 MiB | Running
Memory :  31.79 MiB /  44.50 MiB (71.43%)
Queue  : [1]
CPUs   : [0: 4]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [4] exited with code: 0
```

这是由于我们还没有实现进程退出时内存的释放, 因此此时占用的大小是叠加的, 运行两次之后变成了两倍。

3. 用户程序的内存释放

首先，我们需要在 `stack.rs` 中实现栈的内存释放函数 `clean_up`，代码如下

```rust
pub fn clean_up(
    &mut self,
    mapper: MapperRef,
    dealloc: &mut impl x86_64::structures::paging::FrameDeallocator<Size4KiB>,
) -> Result<(), x86_64::structures::paging::mapper::UnmapError> {
    let count = self.usage;
    if count == 0 {
        warn!("Stack is empty, no need to clean up.");
        return Ok(());
    }

    // The stack range might have more pages mapped if grown,
    // we unmap the inclusive range.
    let range_inclusive = Page::range_inclusive(self.range.start, self.range.end
- 1);
    elf::unmap_range(range_inclusive, mapper, dealloc, true)?;
    self.range = Page::range(self.range.start, self.range.start);

    self.usage = 0;

    Ok(())
}
```

由于我们的内存范围定义是左闭右开的，因此我们要先转化为闭区间 `range_inclusive` 然后调用 `elf::unmap_range()` 进行释放，随后还需要更新栈占用的页面为 0 以及内存的页面范围为一个空区间。

然后我们再在 `proc/vm/mod.rs` 的 `clean_up` 函数中，根据文档加入相关调试信息的输出。

最后，我们需要对 `ProcessVm` 实现 `Drop` trait，调用自己的 `.clean_up()` 函数释放内存，此时 `kill` 函数中调用的 `proc_vm.take()` 在进程退出的时候释放了 `ProcessVm` 的所有权，当 `ProcessVm` 对象没有被任何一个对象持有时，就会自动调用我们实现的 `Drop` trait 中的 `drop` 函数，从而实现了进程退出时的内存释放。

我们可以运行 `factorial`，发现在进程退出之后，使用的内存也被正常释放了！

```
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  1 | #  0 | kernel       |  119398 |   2.0 MiB | Ready
 #  2 | #  1 | /app/sh      |  112529 |  32.0 KiB | Blocked
 #  3 | #  2 | /app/factor~1 |    6841 |  15.3 MiB | Running
Memory :  16.43 MiB /  44.49 MiB (36.94%)
Queue  : [1]
CPUs   : [0: 3]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [3] exited with code: 0
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  1 | #  0 | kernel       |  126421 |   2.0 MiB | Ready
 #  2 | #  1 | /app/sh      |  119547 |  32.0 KiB | Running
Memory :   1.10 MiB /  44.49 MiB ( 2.47%)
Queue  : [1]
CPUs   : [0: 2]
```

4. 内核的内存统计

内核的内存占用包括了内核栈内存的占用以及加载 ELF 文件使用的内存，前者我们已经统计，因此这里只需要实现后者的统计。

我们的方法仍然是在加载内核的 ELF 时就记录下来占用的内存，并记录在 `bootinfo` 中，在初始化内核的时候，把这部分的占用记录到内核的 `ProcessVm` 对象中的 `.code` 和 `.code_usage` 成员中即可。

我们可以观察到，相比于之前的内核占用为 `2.0MiB`，现在变为了 `2.3MiB`，这和 ELF 中 Load 类型的非 .bss 段占用的总内存，即 `0x4b000` Bytes，约为 `0.293MiB` 对应，说明我们成功实现了内核的 ELF 文件的统计！

```
            FileSiz            MemSiz              Flags  Align
LOAD           0x0000000000001000 0xffffff0000000000 0xffffff0000000000
                0x000000000000e084 0x000000000000e084  R      0x1000
LOAD           0x0000000000010000 0xffffff000000f000 0xffffff000000f000
                0x0000000000036bd7 0x0000000000036bd7  R E    0x1000
LOAD           0x0000000000047000 0xffffff0000046000 0xffffff0000046000
                0x0000000000004148 0x0000000000004148  RW     0x1000
```

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |   11394 |  32.0 KiB | Running
 #  1 | #  0 | kernel       |   11424 |   2.3 MiB | Ready
Memory :   1.08 MiB /  44.49 MiB ( 2.43%)
Queue  : [1]
CPUs   : [0: 2]
```

5. 内核栈的自动增长

从上面的部分来看，我们的内核有 `2.0MiB` 的内存占用来自内核栈，这是因为我们为了避免内核栈溢出而分配了大量的内存，因此这部分我们希望把内核栈改为自动增长而不是预先分配大量空间。

这里利用的是 config 里面的 `kernel_stack_auto_grow` 配置项，如果是 `0` 则认为一次性分配了内核栈的所有内存，如果非 0，则表示初始时分配给内核栈的内存。

然后在进行 `map_range` 的时候，需要重新计算需要映射的栈大小以及当前栈底地址，这个可以通过将内核栈最低地址加上未映射的部分得到，代码参考文档给出的进行实现，请见 `boot/src/main.rs`。

随后，我们在 `exception.rs` 中对 `USER_MODE` 位进行判断，并输出对应的日志，同时根据文档给出的代码，在 `lib.rs` 中加入对内核栈缺页异常处理的测试。

可以看到，可以正常通过内核栈增长的测试，并且此时我们 `kernel` 占用的 memory 变成了 `340.0KiB` 说明，我们对内核栈自动扩容的实现符合预期！

```
[INFO] ysos_kernel - Test stack grow.
[INFO] ysos_kernel::interrupt::exceptions - Page fault on kernel at 0xffffff01ffff7df0
[INFO] ysos_kernel - Stack: 0xffffff01ffff7df8
[INFO] ysos_kernel - Stack grow test done.
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+------------------
  counter         |    39000 B | 0x0000111100001470
  mq              |    38928 B | 0x00001111000017e0
  fork_test       |    35464 B | 0x00001111000011d0
  sleep           |    34432 B | 0x00001111000010a0
  hello           |    34336 B | 0x0000111100001020
  dinner          |    34488 B | 0x00001111000014f0
  fish            |    38808 B | 0x0000111100001320
  factorial       |    34880 B | 0x0000111100001490
  sh              |    39672 B | 0x0000111100001960

Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |   34005 |  32.0 KiB | Running
 #  1 | #  0 | kernel       |   34073 | 340.0 KiB | Ready
Memory :   1.09 MiB /  46.45 MiB ( 2.34%)
Queue  : [1]
CPUs   : [0: 2]
[>]
```

6. 用户态堆的实现

我们这部分为了实现用户态对堆的管理，需要实现 `sys_brk` 系统调用给用户态进行堆的使用和管理。这个系统调用支持：如果我们传入 `None`，则会返回目前堆顶的地址，否则会返回 `brk` 之后的新堆顶地址。

首先，我们根据文档和之前系统调用实现的方法，添加 `sys_brk` 的系统调用号 `12`，并在 `lib/src/syscall.rs` 中进行封装，然后再在 `syscall/mod.rs` 的分配器中交给 `service.rs` 下的 `sys_brk` 进行，随后一路向下委托到 `vm/heap.rs` 中的堆自己的 `brk` 函数来进行。

根据文档，首先需要根据传入的 `new_end` 是 `None` 还是一个地址来确定目前是否需要对堆进行操作，

```rust
// FIXED: if new_end is None, return the current end address
let current_end_u64 = self.end.load(Ordering::Relaxed);
let current_end = VirtAddr::new(current_end_u64);
let new_end = match new_end {
    Some(addr) => addr,
    None => return Some(current_end),
};
```

然后，我们要检查传入的地址是否在合法的地址的范围内，

```
// FIXED: check if the new_end is valid (in range [base, base + HEAP_SIZE])
if new_end < self.base || new_end.as_u64() > HEAP_END {
    return None;
}
```

随后，我们根据 `new_end` 的值是让堆的范围扩大还是缩小进行不同的处理，如果是要释放整个堆区，那么就直接进行 `unmap_range`，否则，需要先将 `new_end` 向上对齐（因此不能和 `stack` 直接用 `containing_address` 对齐而要用 `align_up` 向上对齐）到页边界，然后根据是要扩大堆区还是缩小堆区进行 `map_range` 或者是 `unmap_range` 即可，注意我们的堆区范围定义是左闭右开，因此进行 `unmap_range` 的时候要把右边界 `-1`，这部分代码具体请见 `proc/vm/heap.rs`。

最后，我们要记得把堆区的右边界设为我们的新堆顶 `new_end`。

然后，我们要实现堆的释放函数 `clean_up`，和 `stack` 的 `clean_up` 函数类似，唯一的区别就是要把地址向上对齐而不是向下对齐。具体请见 `proc/vm/heap.rs`。

然后我们使用文档给出的代码写一个用户程序 `brk_test` 对 `sys_brk` 系统调用进行测试，同时在其 `Cargo.homl` 中将其使用的 allocator 改成 `brk_alloc`，可以看到，我们现在的用户程序可以正常使用 `sys_brk` 来调整自己的堆大小，同时用户程序初始化的时候，也正常调用了 `sys_brk` 堆用户程序的堆进行扩容初始化

```
[>] brk_test
[DEBUG] ysos_kernel::proc - Spawned process: /app/brk_test#3
[DEBUG] ysos_kernel::proc::vm::heap - heap brk: current_end = 0x200000000000, new_end = 0x200000001ff8,
 current_end_align = 0x200000000000, new_end_align = 0x200000002000, difference = 0x2000
[DEBUG] ysos_kernel::proc::vm::heap - heap brk: current_end = 0x200000001ff8, new_end = 0x200000003ff0,
 current_end_align = 0x200000002000, new_end_align = 0x200000004000, difference = 0x2000
[DEBUG] ysos_kernel::proc::process - Killing process /app/brk_test#3 with ret code: 0
[DEBUG] ysos_kernel::proc::vm - Recycled 17(0.066 MiB) frames, 17(0.066 MiB) frames in total.
Process [3] exited with code: 0
```

7. 思考题

7.1. 当在 Linux 中运行程序的时候删除程序在文件系统中对应的文件，会发生什么？程序能否继续运行？遇到未被映射的内存会发生什么？

若在运行程序时删除程序对对应的文件，什么都不会发生，程序可以继续运行，如果遇到未被映射的内存，则会从磁盘中将对应的代码段或者数据段调入内存，然后继续正常执行直到结束。

这是因为 Linux 中的每个文件实际上是由一个 `inode` 号以及 `DirEntry`，后者实际上是文件名与 `inode` 号的映射，`inode` 中实际保存了文件的大小，权限，文件在磁盘上的位置等信息。当我们删除某个文件的时候，实际上只是删除了 `DirEntry` 的信息，而只有当某个 `inode` 没有任何的 `DirEntry` 指向它（也被称为 Hard Link），且没有任何进程正在打开着这个文件时，Linux 内核才会释放其指向的磁盘位置上的数据。

因此我们删除文件实际上只是删除 `DirEntry`，代码段和数据段等信息还在磁盘上，因此可以正常运行，缺页时也可以正常从磁盘中把页调入内存。

7.2. 为什么要通过 `Arc::strong_count` 来获取 `Arc` 的引用计数？查看它的定义，它和一般使用 `&self` 的方法有什么不同？出于什么考虑不能直接通过 `&self` 来进行这一操作？

因为我们在进行 `fork` 的时候，子进程的堆是和父进程共享的，子进程退出的时候，会调用当前进程的堆的 `clean_up` 函数，如果此时 `clean_up` 将当前进程的堆提前释放，则会导致父进程出错，因此我们要先用 `Arc::strong_count` 获取当前堆的强引用计数，来保证堆内存不会提前释放。

```rust
pub fn strong_count(this: &Self) -> usize {
    this.inner().strong.load(Relaxed)
}
```

这个方法是一个静态方法（参数是 `this: &Self` 而不是以 `self` 开头的），不可以通过成员函数的方式进行调用。这样设计的原因是：`Arc<T>` 实现了 `Deref` 隐式解引用，即外壳 `Arc` 对内部成员函数透明。因此如果 `T` 内部实现了 `.strong_count()` 成员函数，那么在 `Arc<T>` 类型下调用 `.strong_count()` 就不能得到预期的行为，因此这里设计为静态方法。

7.3. `bootloader` 加载内核并为其分配初始栈区时，至少需要多少页内存才能保证内核正常运行？

- 尝试逐渐增大内核的栈区大小，观察内核的运行情况，对于不能正常启动的情况，尝试分析可能的原因。
- 提示：内核实现缺页中断的处理时，依赖于哪些子系统？报错是什么？什么子系统可能会导致对应的问题？

经过测试，至少需要 4 页内存才能保障内核的正常运行，当初始栈区设为 3 页时，会在如下输出后重启，

```
[INFO] ysos_kernel::memory::gdt - Kernel IST Size  :  16.000 KiB
[INFO] ysos_kernel::memory::gdt - GDT Initialized.
[INFO] ysos_kernel::memory - Physical Memory    :  95.625 MiB
[INFO] ysos_kernel::memory - Free Usable Memory :  46.477 MiB
[INFO] ysos_kernel::memory - Frame Allocator initialized.
[INFO] ysos_kernel::memory::user - User Heap Initialized.
```

设为 2 页及以下的时候，会在如下输出后重启，

```
[INFO] ysos_kernel::utils::logger - Logger Initialized.
[INFO] ysos_kernel::memory::address - Physical Offset  : 0xffff800000000000
[INFO] ysos_kernel::memory::gdt - Privilege Stack  : 0xffffff000084c550-0xffffff000084d55
0
[INFO] ysos_kernel::memory::gdt - Double Fault IST : 0xffffff000084d550-0xffffff000084e55
0
[INFO] ysos_kernel::memory::gdt - Page Fault IST   : 0xffffff000084e550-0xffffff000084f55
0
[INFO] ysos_kernel::memory::gdt - Clock IST        : 0xffffff000084f550-0xffffff000085055
0
[INFO] ysos_kernel::memory::gdt - Syscall IST       : 0xffffff0000850550-0xffffff00008515
50
[INFO] ysos_kernel::memory::gdt - Kernel IST Size  :  16.000 KiB
[INFO] ysos_kernel::memory::gdt - GDT Initialized.
[INFO] ysos_kernel::memory - Physical Memory    :  95.625 MiB
[INFO] ysos_kernel::memory - Free Usable Memory :  46.480 MiB
```

上述两者都是由于中断处理未初始化，从而引发了 triple fault，前者是因为我们的 IDT 需要占用一定的空间因此页数不够时会 page fault，后者是因为在 `Frame Allocator` 初始化的时候，其中的 `create_frame_inter` 函数中使用了 `.clone`，此处是将其进行深拷贝放入当前函数的栈，即内核栈中，从而引发了栈溢出 page fault。

其次，由于缺页中断处理依赖于中断子系统以及内存子系统，因此在 `Interrupts Initialized.` 之前和缺页异常栈 `Page Fault IST` 初始化之前，缺页中断都不能正常对内核栈进行扩容，缺页异常由于找不到对应的中断处理程序或者没有内核栈，从而再次抛出异常，引发 triple fault 导致操作系统没有任何提示而直接不断重启。

7.4. 尝试查找资料，了解 `mmap`、`munmap` 和 `mprotect` 系统调用的功能和用法，回答下列问题：

- `mmap` 的主要功能是什么？它可以实现哪些常见的内存管理操作?

其主要功能是可以将一个文件或者一段匿名内存等映射到进程的虚拟地址空间中。可以实现文件映射，程序直接通过读写指针访问文件；也可以以共享内存的方式实现进程间通信。

- `munmap` 的主要功能是什么？什么时候需要使用 `munmap`？

用于释放由 `mmap` 建立的内存映射。当我们不需要访问该文件映射，或者希望释放匿名内存，进程间的共享内存等的时候需要使用 `munmap` 进行释放，避免内存的泄漏。

- `mprotect` 的主要功能是什么？使用 `mprotect` 可以实现哪些内存保护操作?

`mprotect` 用于更改某一段由 `mmap` 映射好的虚拟内存的的保护属性。如可以更改为只读 `PROT_READ` 防止文件被修改，还有 `PROT_WRITE` 可写属性以及 `PROT_EXEC` 可执行属性等。也可以使用 `PROT_NONE` 清空权限，用来防止栈越界等问题。

- 编写 C 程序，使用 mmap 将一个文件映射到内存中，并读写该文件的内容。

代码如下，具体的说明已经写在注释中

```c
int main() {
    const char *filepath = "test.txt";

    // O_RDWR: 以读写方式打开文件
    // O_CREAT: 如果文件不存在则创建文件
    // 0666: 创建文件的权限，表示所有用户都具有读写权限
    int fd = open(filepath, O_RDWR | O_CREAT, 0666);

    // 设置文件大小为 100 bytes
    ftruncate(fd, 100);

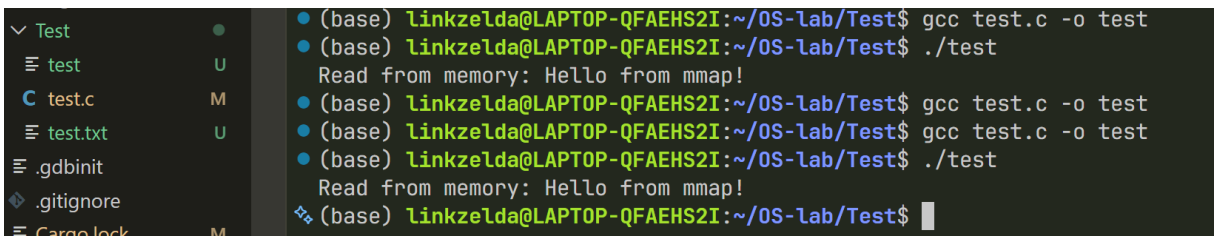
    // 将文件映射到内存
    // PROT_READ | PROT_WRITE: 映射区域可读可写
    // MAP_SHARED: 映射区域的修改会反映到文件中
    // 也可也设置为 MAP_PRIVATE: 映射区域的修改不会反映到文件中，相当于 COW 机制
    // NULL: 由系统选择映射地址
    // 0: 从文件的开头开始映射
    // 返回映射的地址，如果失败则返回 MAP_FAILED
    char *mapped = mmap(NULL, 100, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    // 写文件内容
    strcpy(mapped, "Hello from mmap!");

    // 读文件内容
    printf("Read from memory: %s\n", mapped);

    // 取消映射与关闭文件
    munmap(mapped, 100);
    close(fd);
    return 0;
}
```

我们编译运行这个程序，可以看到，文件被创建，并成功读写内容。



- 思考：文件内容什么时候会被写入到磁盘?

使用 `mmap` 写入的文件内容，实际上是写入 Cache，让对应的页的 dirty 位置 1，会由操作系统的内核线程 `pdflush` 定期扫描，将 dirty 位为 1 的页写回磁盘；在要进行页面调出的时候，也会写入磁盘；用户程序也可以使用 `msync()` 系统调用强制把页面内容立即写入磁盘。

- 综合考虑有关内存、文件、I/O 等方面的知识，讨论为什么 `mmap` 系统调用在现代操作系统中越来越受欢迎，它具有哪些优势?

因为 `mmap` 系统调用是直接把内核中的 Cache 映射到了用户的虚拟内存中，常规的 `read/write` 会把数据拷贝到用户的 Buffer，前者可以节省 CPU 开销；其次，我们映射到用户的虚拟内存之后，直接用指针访问文件即可，避免了使用 `read/write` 系统调用的内核态切换，且操作较为简单。且在进行进程间共享内存的时候，不需要复制多份到进程缓冲区，而是直接共享物理页，节省了内存的开销。

8. 加分项

我们在文档给出的实现参考的基础上加上一个 `total_size` 成员，用来记录当前用户态的堆大小，然后初始化以及修改的时候原子性地对 `total_size` 进行修改（从而保证我们地用户程序在多线程下可以正常按顺序进行扩容)。

对于 `extend` 函数，我们直接尝试更新 `total_size`，然后调用 `sys_brk` 映射需要扩容堆空间地址，以及使用 `LockedHeap` 自带的 `extend` 进行内存分配器的堆扩容，我们这里使用 `loop` 是为了循环尝试按顺序修改 `total_size` 并修改，

```rust
pub unsafe fn extend(&self, layout: Layout) -> bool {
    // FIXED: extend heap size with `brk` system call
    //        return false if failed or reach the max size (8 MiB suggested)
    let required = layout.size();

    loop {
        // use loop to retry if compare_exchange failed
        let current_size = self.total_size.load(Ordering::SeqCst);
        let new_size = current_size + required;
        if new_size > HEAP_MAX_SIZE {
            return false;
        }
        if self
            .total_size
            .compare_exchange(current_size, new_size, Ordering::SeqCst,
Ordering::SeqCst)
            .is_ok()
        {
            let head_start = sys_brk(None).unwrap();
            let head_new_end = head_start + required;
            let ret = sys_brk(Some(head_new_end)).expect("Failed to extend
heap");
            if ret == head_new_end {
                unsafe { self.allocator.lock().extend(required) };
                return true;
            } else {
                // rollback the total size if failed
                self.total_size.fetch_sub(required, Ordering::SeqCst);
                return false;
            }
        }
    }
}
```

随后，对于 `alloc`，我们就先尝试使用 `LockedHeap` 进行 `alloc`，如果不成功（得到了空地址），则进行堆的扩容，

```rust
// FIXED: if alloc failed, ptr is null
// FIXED: try to extend heap size, then alloc again
if ptr.is_null() && self.extend(layout) {
    ptr = self.allocator.alloc(layout);
}
```

其余部分代码请见 `allocator/brk.rs`。

然后我们进行测试，我们进行测试的代码如下，我们初始时设置的堆大小是 8 KiB，然后我们在用户程序中通过向 `Vec` 中加入元素以及用 `Box` 在堆上声明一个很大的数组来测试堆是否可以自动扩容，这里为了防止被编译优化，输出了 `large_box` 的首尾元素。

```rust
fn main() -> isize {
    println!("Testing Automatic Heap Expansion...");

    let mut large_vec = Vec::new();
    for i in 0..10000 {
        large_vec.push(i);
    }

    let mut large_box = Box::new([0u8; 1024 * 1024]);

    large_box[0] = 42;
    large_box[1024 * 1024 - 1] = 84;

    // avoid optimization
    println!(
        "Large Box first element: {}, last element: {}",
        large_box[0],
        large_box[1024 * 1024 - 1]
    );

    println!(
        "Vec length: {}, Box alloc size: {}",
        large_vec.len(),
        large_box.len()
    );
    println!("Heap test PASSED!");

    0
}
```

可以从日志中看到，确实堆实现了自动扩容，也符合 `Vec` 的扩容表现（每次扩容大小是前一次的两倍），以及 `Box` 一次性 alloc 了 1 MiB 的堆空间，说明我们的堆自动扩容实现符合预期！

```
[>] brk_test
[DEBUG] ysos_kernel::proc - Spawned process: /app/brk_test#3
[DEBUG] ysos_kernel::proc::vm::heap - heap brk: current_end = 0x200000000000, new_end = 0x200000001ff8,
 current_end_align = 0x200000000000, new_end_align = 0x200000002000, difference = 0x2000
[DEBUG] ysos_kernel::proc::vm::heap - heap brk: current_end = 0x200000001ff8, new_end = 0x200000003ff0,
 current_end_align = 0x200000002000, new_end_align = 0x200000004000, difference = 0x2000
[DEBUG] ysos_kernel::proc::process - Killing process /app/brk_test#3 with ret code: 0
[DEBUG] ysos_kernel::proc::vm - Recycled 17(0.066 MiB) frames, 17(0.066 MiB) frames in total.
Process [3] exited with code: 0
```